Insurance Premium Prediction

End-to-End Machine Learning & Deployment

Task 1 · ML Classification | AlFido Tech Internship | Python 3.14 | scikit-learn | Streamlit

1. Overview

An end-to-end machine learning project that predicts **annual medical insurance charges** from demographic and lifestyle factors. It covers the complete ML lifecycle: exploratory data analysis, leakage-free preprocessing, training and tuning of four regression models, automated best-model selection, and deployment as an interactive Streamlit web application.

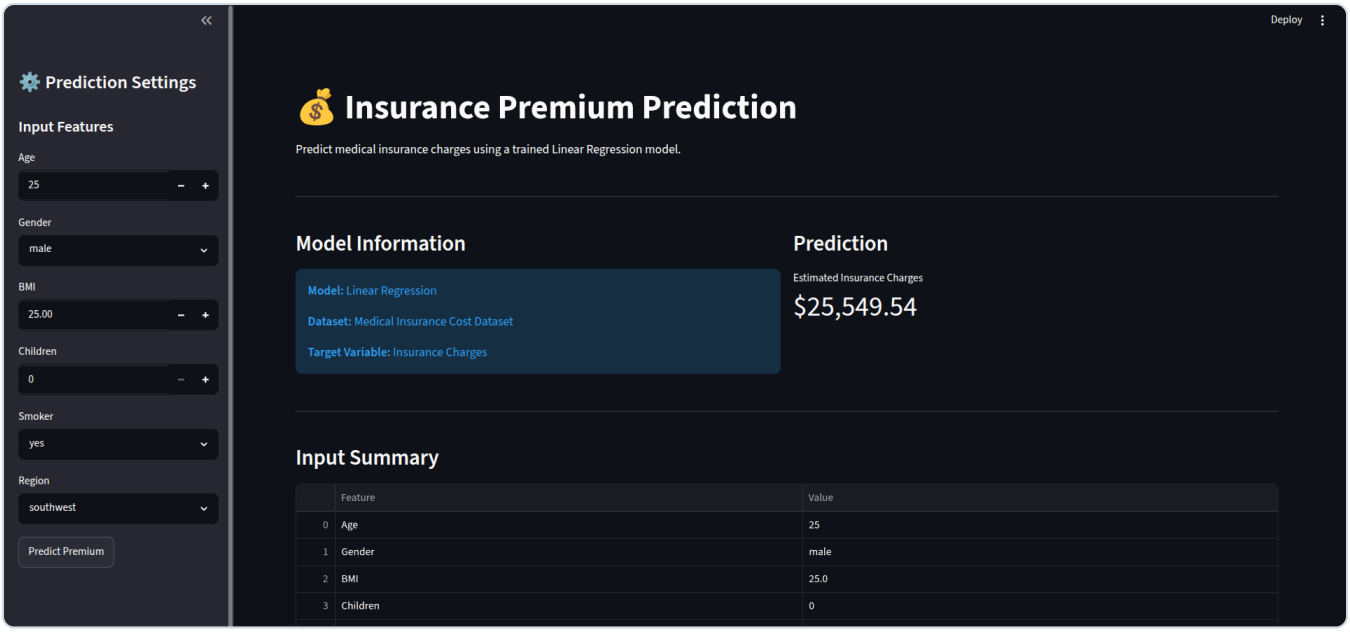
Problem type: Supervised regression · **Target:** Insurance charges (USD) · **Best model:** Linear Regression (R^2 = 0.796)
Dataset: Medical Insurance Cost — 1,338 records, 6 features · **Deliverable:** Live Streamlit app + 4 serialized model pipelines

2. Links & Artifacts

Resource	Link
Live Streamlit App	https://rr-insurance-predictor.streamlit.app/
GitHub Repository	https://github.com/Demon-diablo/insurance-premium-prediction
Jupyter Notebook	`notebooks/insurance_eda__multi_model.ipynb` (in repo)
Google Drive (models/data)	*<paste Drive link if hosting large files>*

Note: model `.pkl` files (~3.5 KB each) and the dataset (54 KB) are small and committed directly to the repo, so a Drive link is optional for this task.

3. Application Screenshot



Streamlit app — model selector, input sidebar, live prediction, and model-comparison dashboard.

4. Results

All four models were tuned with 5-fold `GridSearchCV` and evaluated on a held-out test set. Linear Regression achieved the best performance:

Model	R^2 Score	RMSE (USD)	MAE (USD)
Linear Regression ★	**0.796**	**5,940**	**4,069**
Ridge	0.795	5,947	4,077
Lasso	0.795	5,952	4,076
ElasticNet	0.794	5,967	4,100

> All four models perform comparably — regularization (Ridge/Lasso/ElasticNet) offers minimal gains, indicating low multicollinearity among features. The app auto-highlights and pre-selects the best model via `metrics.json`.

5. Approach & Explanation

Pipeline

- **Data cleaning & EDA** — removed 1 duplicate (1,338 → 1,337 rows); analyzed distributions, correlations, and outliers. Smoking status emerged as the dominant cost driver.
- **Preprocessing** — a scikit-learn `Pipeline` + `ColumnTransformer` applies `StandardScaler` to numeric features and `OneHotEncoder` to categoricals, fitted only on training data to prevent leakage.
- **Modeling** — four linear regression variants trained; Ridge/Lasso/ElasticNet tuned via `GridSearchCV` (5-fold CV).

- **Evaluation & selection** — scored on R^2, RMSE, MAE; best model and all metrics written to `metrics.json`, which the app reads on load.
- **Deployment** — Streamlit app with model selector, live prediction, dataset explorer, visualizations, and a model-comparison dashboard.

Key Code — Prediction Pipeline (`app/app.py`)

```
# Serialized sklearn Pipeline handles scaling + encoding internally
def make_prediction(model, age, sex, bmi, children, smoker, region):
    input_df = pd.DataFrame([{
        "age": age, "sex": sex, "bmi": bmi,
        "children": children, "smoker": smoker, "region": region
    }])
    return model.predict(input_df)[0]

# Best model auto-selected from metrics.json on app load
default_index = model_options.index(best_model_name) \
    if best_model_name in model_options else 0
```

6. Tech Stack

Category	Technologies
Language	Python 3.14
Machine Learning	scikit-learn (Pipeline, ColumnTransformer, GridSearchCV)
Data Processing	pandas, NumPy
Visualization	Matplotlib, Seaborn
Web / Deployment	Streamlit
Serialization	Joblib

7. Reproducibility — Environment & Commands

Exact package versions (from `requirements.txt`) for a reproducible run:

Package	Version	Package	Version
python	3.14.4	matplotlib	3.11.0
scikit-learn	1.9.0	seaborn	0.13.2
pandas	3.0.3	streamlit	1.58.0
numpy	2.4.6	joblib	1.5.3

Exact commands

```
# 1. Clone and install
git clone https://github.com/Demon-diablo/insurance-premium-prediction
cd insurance-premium-prediction
python3 -m venv .venv
source .venv/bin/activate            # Windows: .venv\Scripts\activate
pip install -r requirements.txt

# 2. (Optional) Retrain — regenerates the 4 .pkl files + metrics.json
jupyter notebook notebooks/insurance_eda__multi_model.ipynb

# 3. Launch the Streamlit app
streamlit run app/app.py
```

Project structure:
`app/app.py` — Streamlit application · `notebooks/insurance_eda__multi_model.ipynb` — EDA + training (45 code cells)
`models/*.pkl` — 4 trained pipelines + `metrics.json` · `data/raw/insurance.csv` — dataset · `requirements.txt` — pinned dependencies